Memory Management

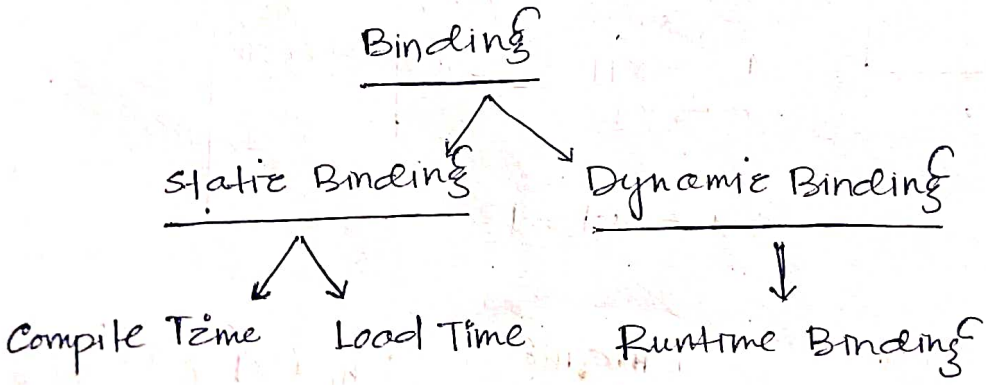
Memory consists of large array of bytes or words each with its own address.

Interaction of memory is achived through a sequence of read and write operation to a specific location of memory.

CPU fetches instruction from memory according to the value of the program counter. The instruction is then decoded and may cause operands to be fetched from memory.

After the instruction has been executed on the operands the result is stored back in memory.

Address Binding :



A program generally resides on a disk as a binary executable file. For execution the program must be brought into memory and placed within a process.

Addresses in the source program are generally symbolic. A compiler will bind these symbolic addresses to relocatable addresses.

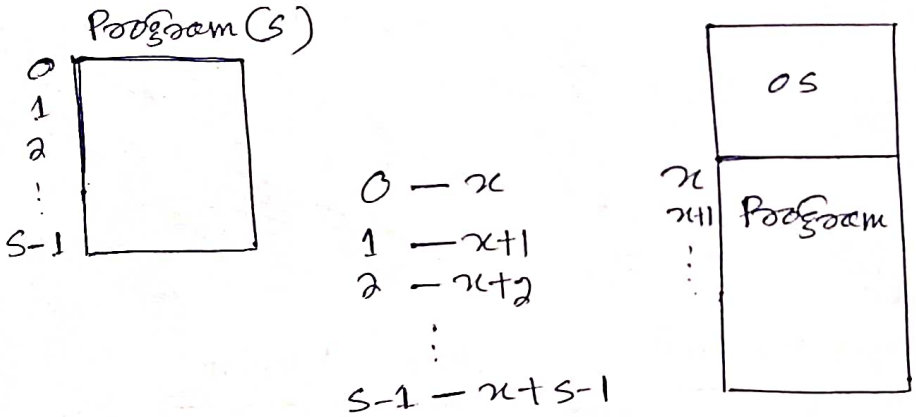
The linker or loader will intern bind the relocatable addresses to absolute addresses.

The binding of instructions and data to memory addresses can be done in the following ways:

(1) **Compile time:** If you know at compile time where the process will reside in memory then absolute code can be generated.

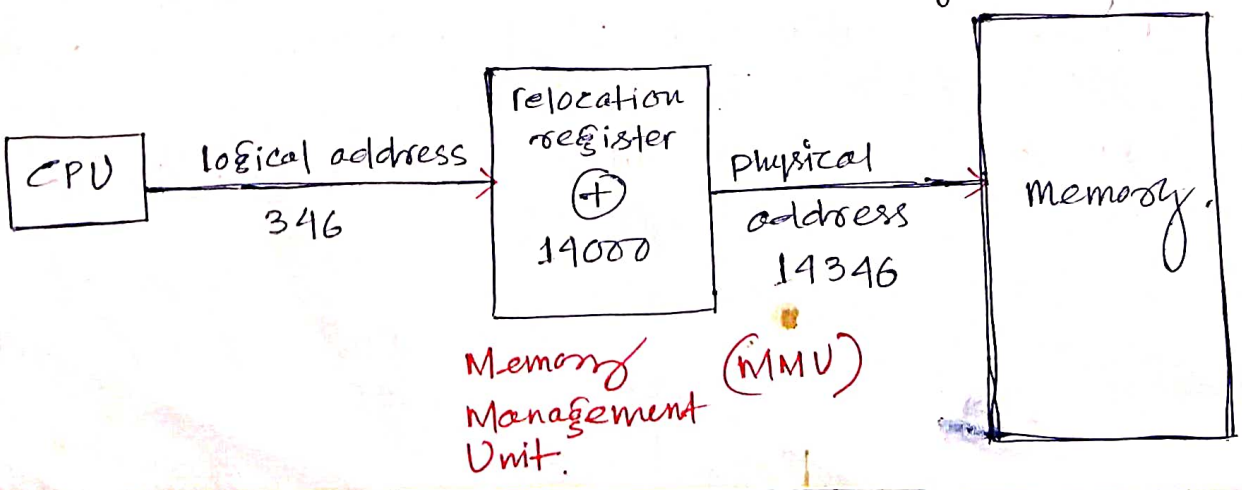
(2) **Load time:** If it is not known at compile time where the process will reside in memory, then the compiler must generate relocatable code. In this case binding is delayed untill load time.

(3) **Execution time:** If the process can be moved during its execution from one memory segment to another, then binding must be delayed until run time



Logical Address or Relative Address → Binding → Physical Address or Absolute Address.

An address generated by CPU is commonly referred as to a logical address where as an address seen by memory unit is called physical address.



Contiguous Memory Allocation :

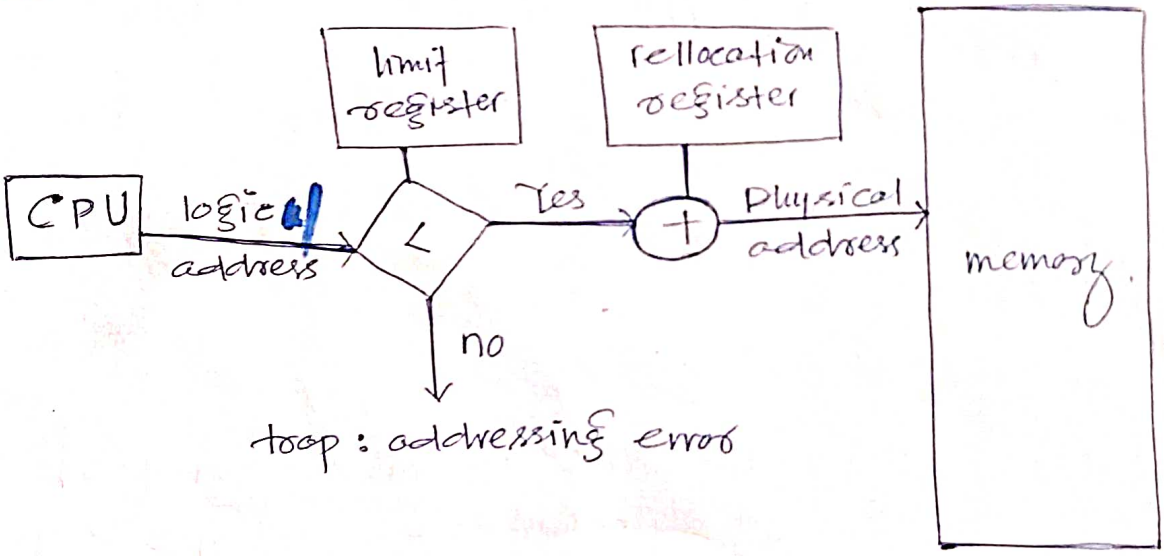
The main memory must accommodate both the operating system and the various user process. So the memory is usually divided into two partitions : one for the resident operating system and one for the user processes.

The operating system is generally stored in low memory.

In multiprogramming environment several user processes reside in memory at the same time.

In contiguous memory allocation, each process is contained in a single contiguous section of memory.

Protection :



Protection of memory is provided by using a relocation register with a limit register.

The relocation register contains the value of the smallest physical address and the limit register contains the range of logical addresses.

With relocation and limit registers each logical address must be less than the limit register; and MMU maps the logical address dynamically by adding the value in the relocation register.

In case of any illegal address operating system will generate an error message by generating a trap.

Memory Allocation :

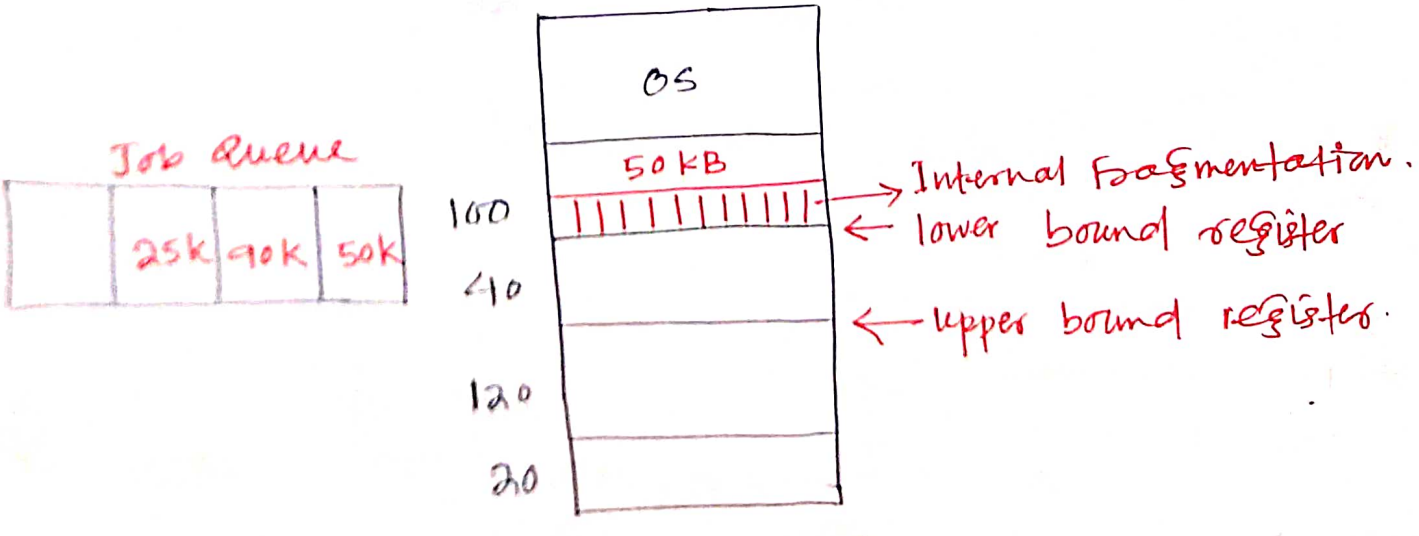
To allocate memory to many programs, memory is divided into number of partitions to make degree of multiprogramming more than one.

When a partition is free a program is selected from Job queue and loaded into that partition. When the program terminates its partition is made available for another program.

Multiple Partition
- ① Fixed Partition. Multiprogramming with Fixed no of Tasks. (MFT)
- ② Variable Partition Multiprogramming with Variable no of Tasks (MVT)

① MFT :

In this case the partition size are fixed and donot change when the system runs. When a job enters the system they are put into a job queue. The scheduler takes into account the memory requirement of each job and available partition size to determine which job will allocated to which partition.

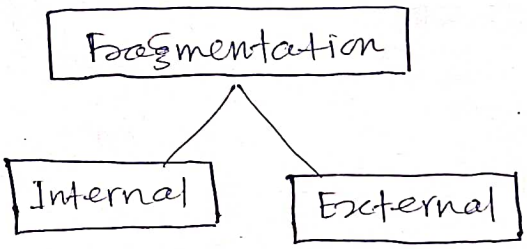


Available free partitions can be allocated to different processes based on different allocation principles.

① **First fit:** Allocate the first hole/partition that is big enough. Searching can start either at the beginning of the partition or where the previous first fit search ended

② **Best fit:** Allocate the smallest partition that is big enough. We must search the entire list, unless the list is ordered by size.

Performance Criteria:



Amount of unused memory within a partition is known as internal fragmentation.

The amount of unused memory spread over multiple partitions is known as external fragmentation.

The MFT scheme suffers from both internal as well as external fragmentation.

Selecting the proper partition sizes are very difficult.

② MVT:

The OS keeps a table indicating which part of memory are available and which are occupied. Initially entire memory is available for user and that is considered as a large block of available memory called a hole.

When a job arrives the job scheduler will search for a hole that is large enough for that job. If such a hole is found then it is allocated as much as needed by the job and rest memory is available to satisfy the future needs.

When a job terminates, its memory is placed into set of holes. If the new hole is adjacent to another hole then they are merge together to make a large hole.

Memory is allocated to jobs until the memory requirement of the next job cannot be satisfied

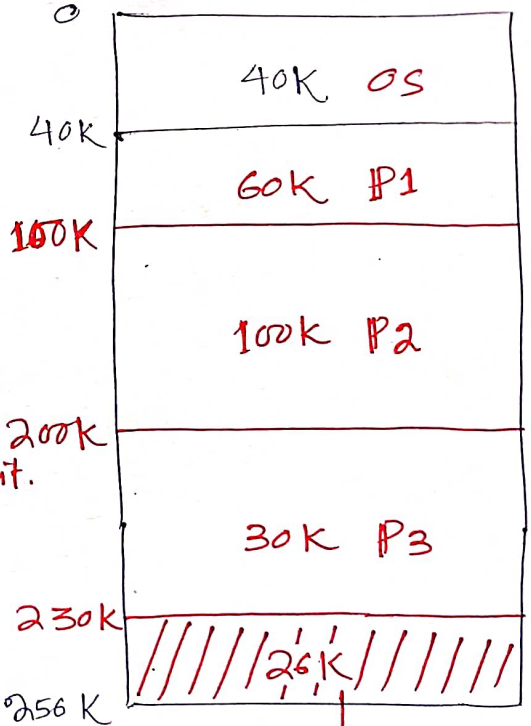
There will be no internal fragmentation but there can be external fragmentation.

External fragmentation can be avoided by shuffling the memory contents to place all free memory together in one large block. This is called compaction.

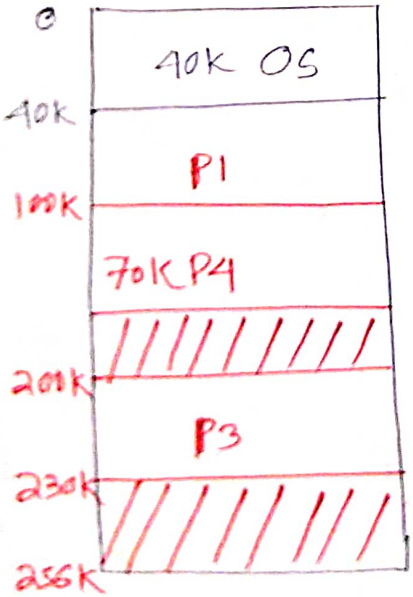
Compaction is possible only if the system uses dynamic binding with the help of base and limit register.

Job	Memory	Time
P1	60K	10
P2	100K	5
P3	30K	20
P4	70K	8
P5	50K	15
P6	60K	9

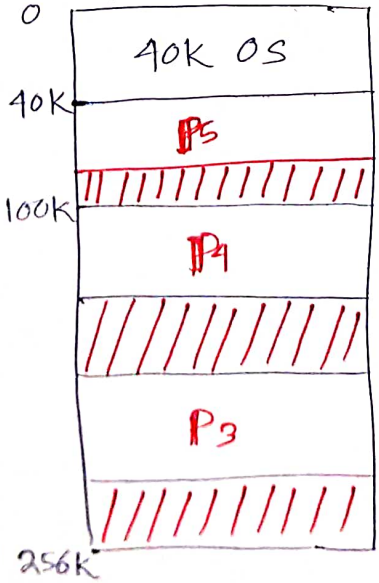
(1)



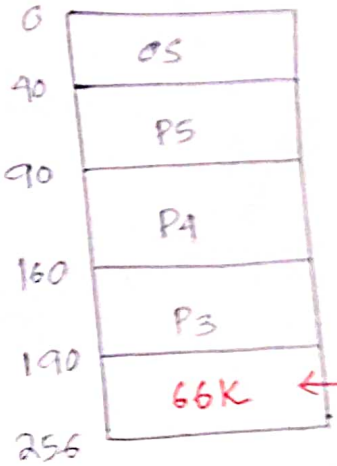
(2) After 5 Time Unit



(3) After 5 Time Unit.



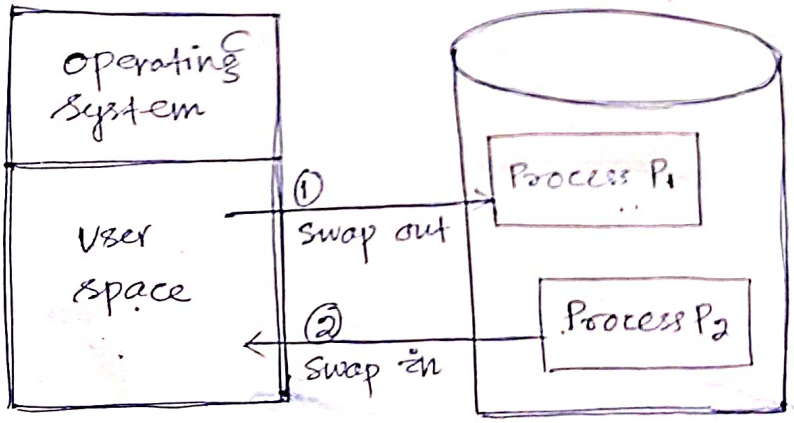
- After Compaction :



Swapping :

A process can be swapped temporarily out of memory to a backing store and then brought back into memory for continued execution.

Normally a process that is swapped out will be swapped back into the same memory space it is occupied previously.



Swapping requires a backing store. The backing store is commonly a fast disk.

Whenever the CPU scheduler decides to execute a process, it calls the dispatcher. The dispatcher checks to see whether the next process in the queue is in memory.

If it is not and there is no free memory region, the dispatcher swaps out a process currently in memory and swaps in the required process.

Then it reloads the registers and transfer control to the selected process.

1. Segmentation :

Segmentation is a memory management scheme that supports user view of memory.

A logical address space is collection of segments. Each segment has a name and length.

The addresses specify both the segment name and offset within the segment.

The user therefore specifies each address by two quantities : (a segment name and an offset.)

For simplicity of implimentation, segments are numbered and referred by the segment number rather than segment name.

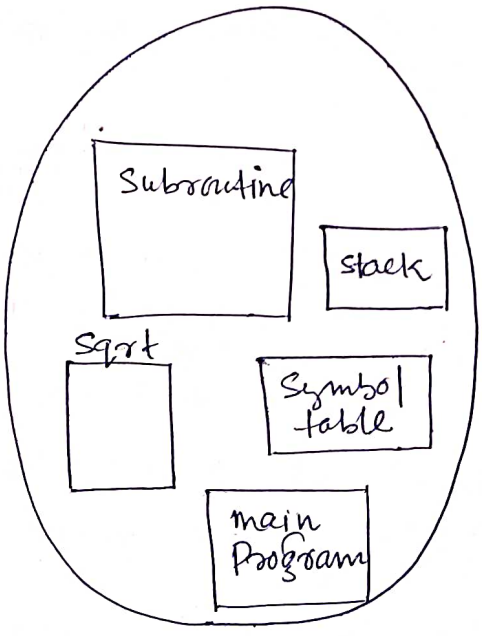
The logical address consists of two-tuples.

< segment-number, offset >

Normally when a program is compiled, the compiler automatically constructs segments reflecting the input program.

A compiler might create separate segments for the following.

(1) The code
(2) Global variables.
(3) The heap.
(4) The stack.
(5) The standard c library



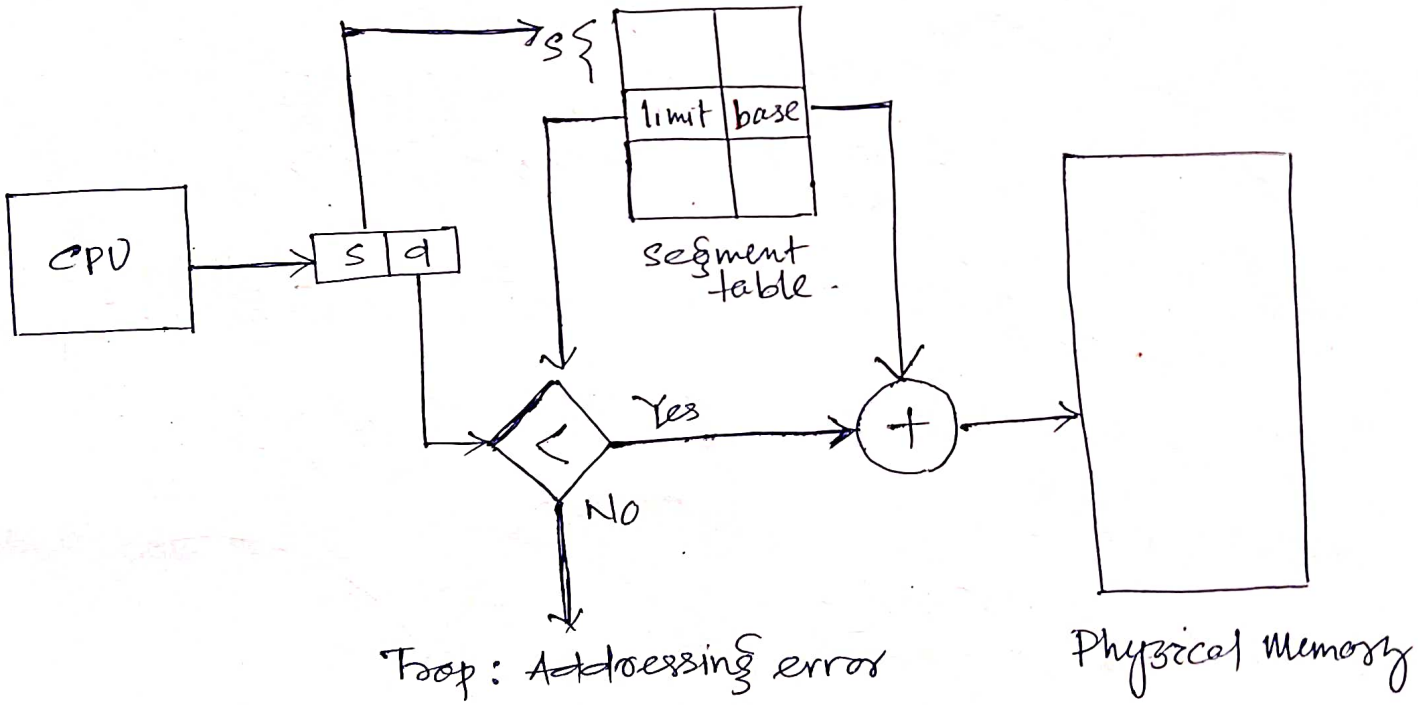
[Users view of program]

Segmentation Hardware :

Although user can refer to objects in the program by a two dimensional address, the actual physical memory is one-dimensional sequence of bytes.

The mapping of two-dimensional address into one-dimensional physical address is performed using ~~segment table~~ segment table.

Each entry in the segment table has a segment base and a segment limit. The segment base contains the starting physical address where the segment resides and limit specifies the length of the segment.



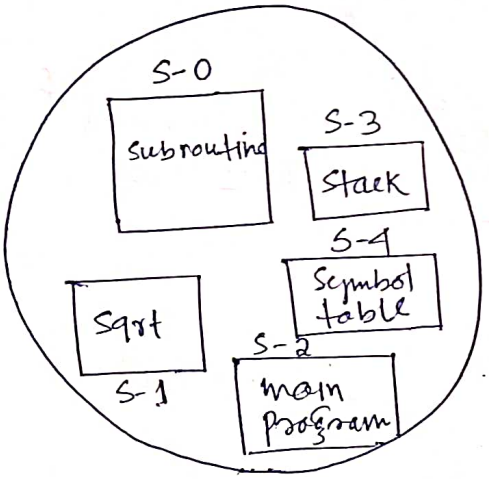
A logical address consists of two parts : segment no. s, and offset into that segment d.

The segment number is used as an index to segment table.

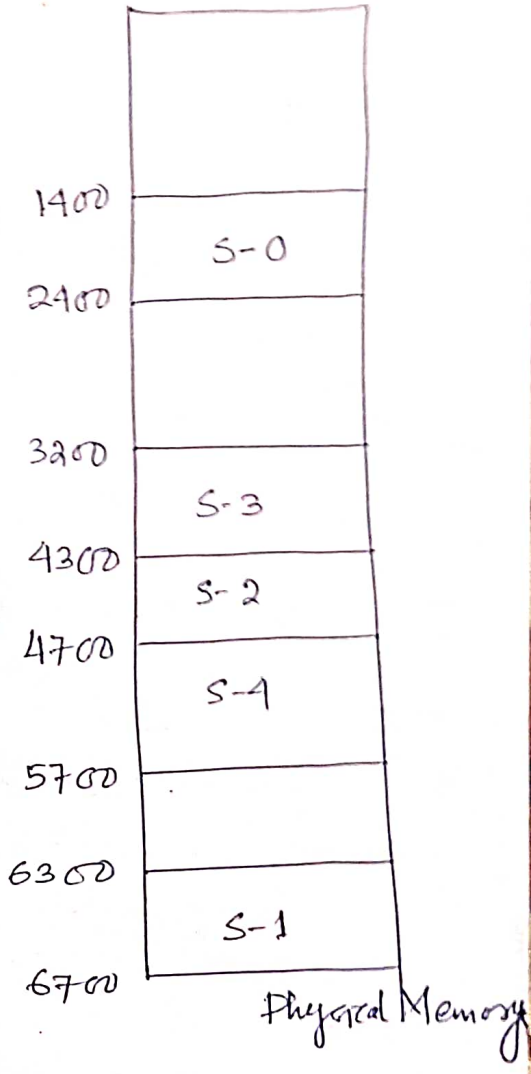
The offset d must be between 0 and segment limit. If it is not we trap to the operating system.

When an offset is legal it is added to the segment base to produce the address in physical memory of desired byte.

Example of Segmentation :



	limit	base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700



② Paging :

Paging is a memory management scheme that permits the physical address space of a process to be non contiguous.

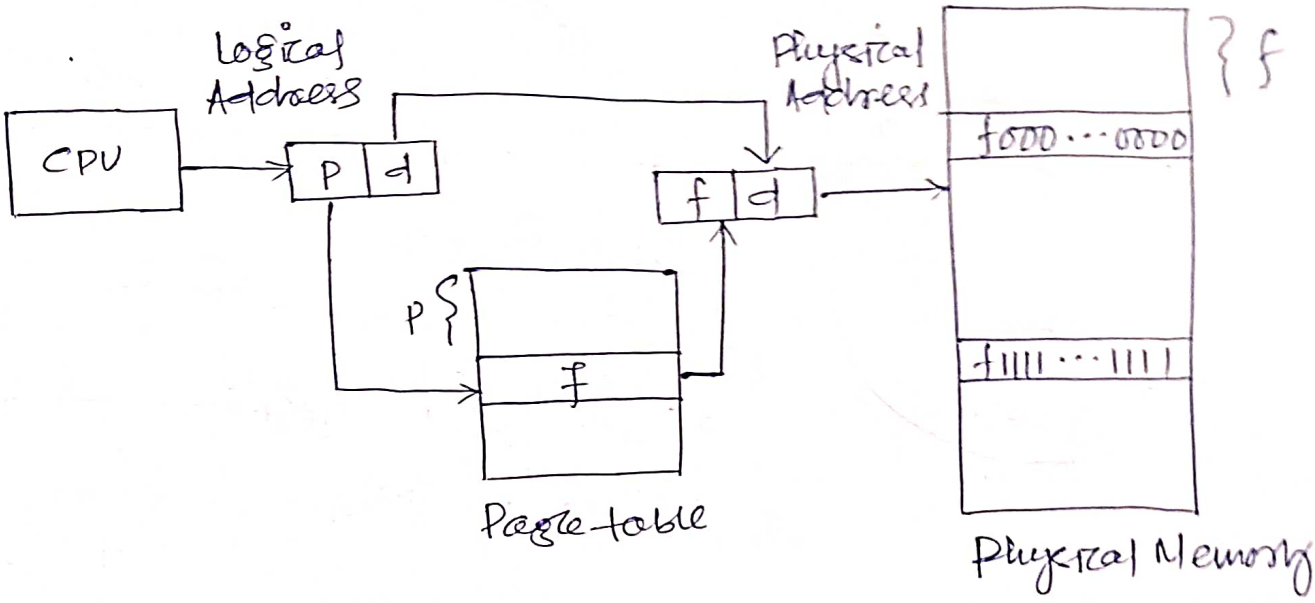
In paging physical memory is divided into fixed sized blocks called frames and logical memory into block of same size called pages.

When a process is to be executed its pages are loaded into any available memory frames from the backing store.

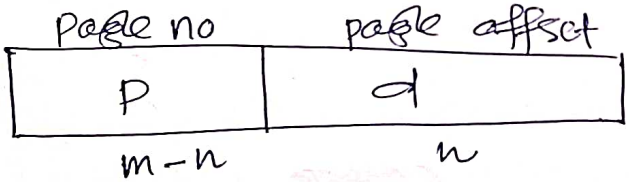
Every address generated by CPU is divided into two parts : a page number (p) and page offset (d).

The page number is used as an index into a page table. The page table contains the base address of each page in physical memory.

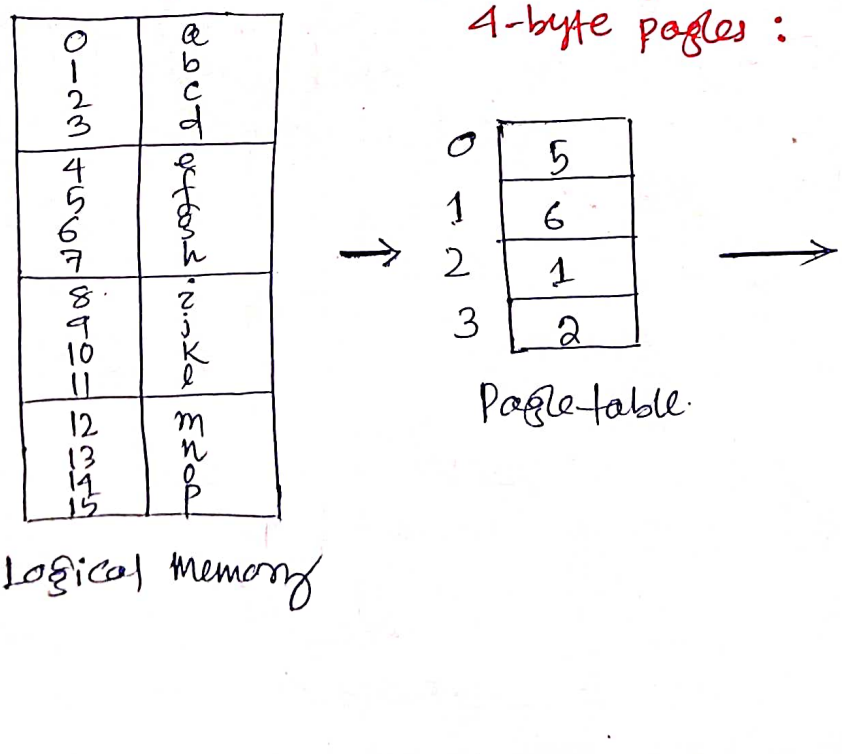
The base address is combined with the page offset to define the physical memory address that is sent to memory unit.



The size of the page typically power of 2. If the size of logical address space is 2^m and page size is 2^n then high-order m-n bits of a logical address represents page number and low-order n bit represents page offset.



Paging example for 32 byte memory with 4-byte pages:

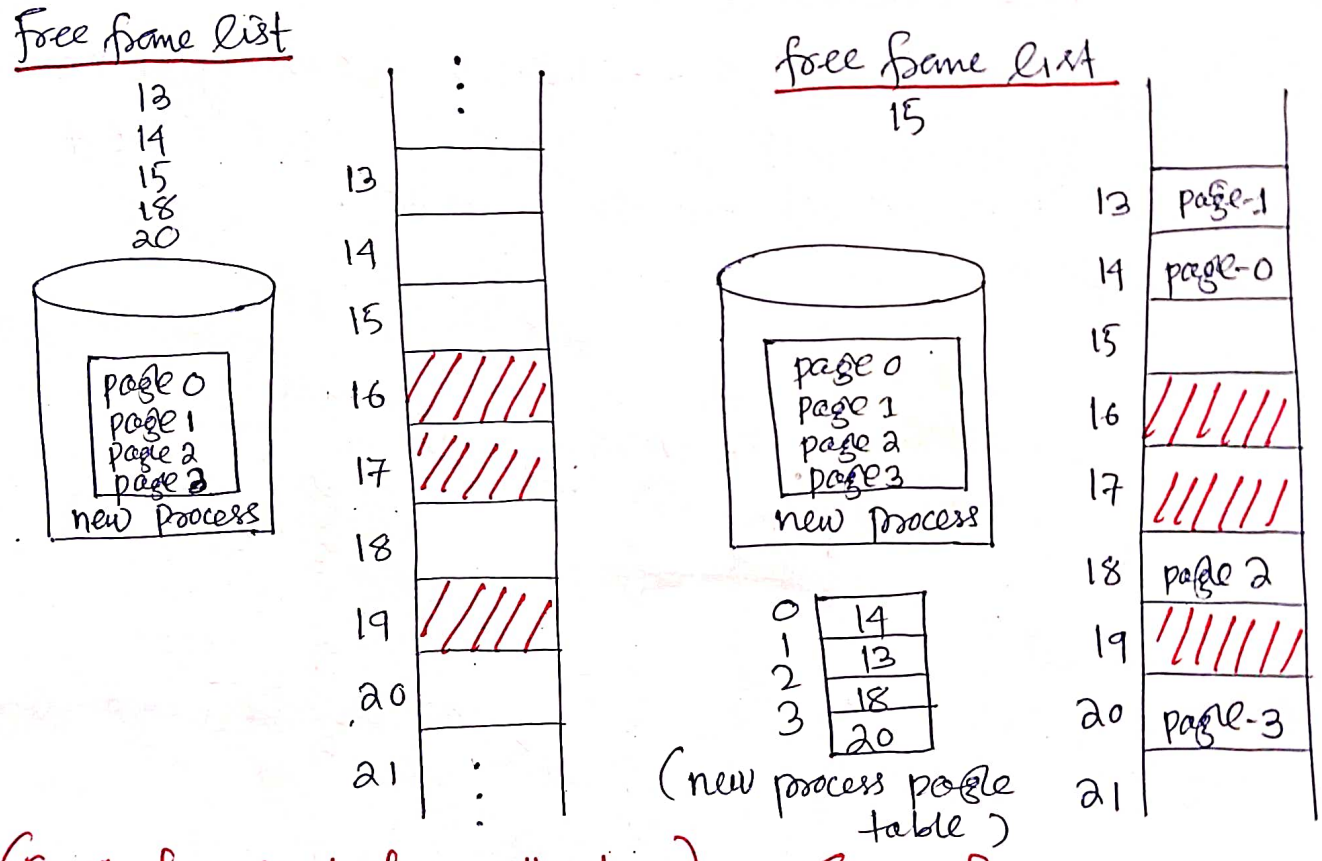


When we use paging we have no external fragmentation. However we have some internal fragmentation.

When a process arrives in the system to be executed its size expressed in pages. Each page of the process needs one frame.

If a process requires n pages, at least n frames must be available in memory.

If n frames are available, they are allocated to this arriving process.



(Free frames before allocation) (Free frames after allocation)

Protection :

Memory protection in paging environment can be accomplished by protection bit associated with each frame.

One bit can define a page to be read-write or read only. An attempt to write to read only page cause a hardware trap to the operating system.

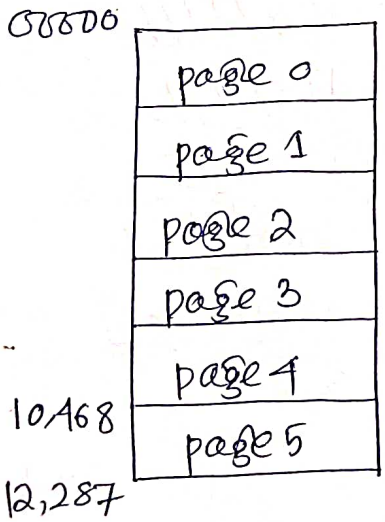
One additional bit is attached to each entry in the page table : a valid-invalid bit.

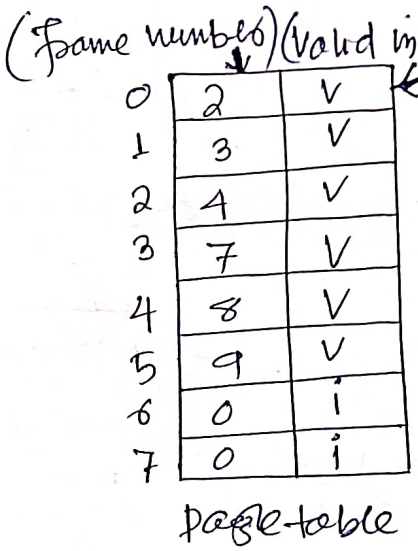
When this bit is set to valid the associated page is legal. When the bit is set to invalid the page is not in the process's logical address space.

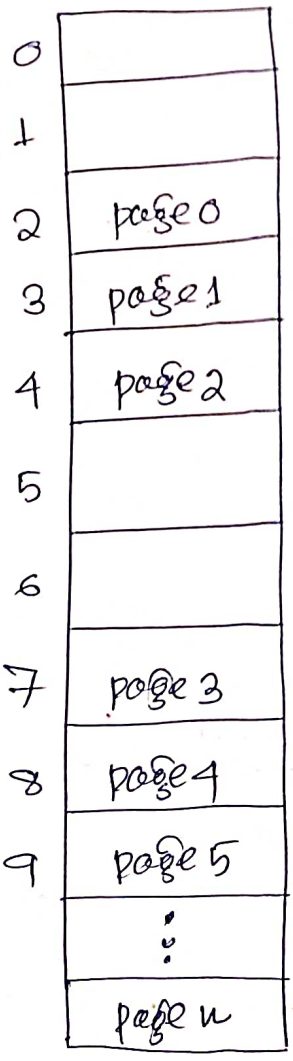
The operating system sets this bit for each page to allow or disallow access to the page.

Example : 14 bit address space (0 to 16383)
Program use only (0 to 10468)
Page size 2KB

Addresses in pages 0, 1, 2, 3, 4 and 5 are mapped normally. Any attempt to generate an address in page 6 or 7 will find valid-invalid bit to invalid.







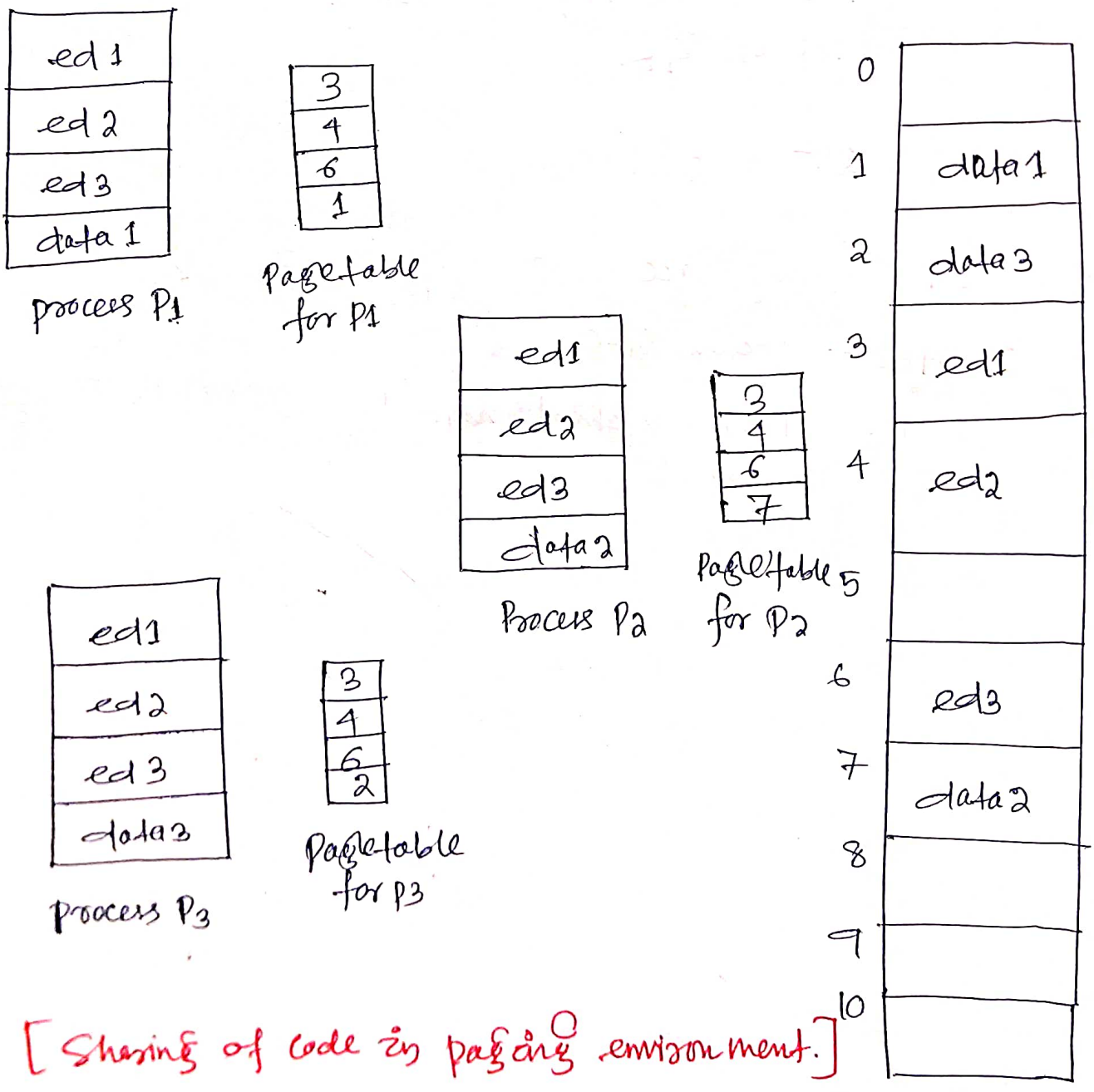
Shared Pages :

An advantage of paging is the possibility of sharing common code.

If the code is reentrant code (pure code) it can be shared. Reentrant code is non-self-modifying code; it never changes during execution.

Thus two or more process can execute the same code at the same time.

Each process has its own copy of registers and data storage to hold data for the process's execution.



[Sharing of code in paging environment.]

Virtual Memory:

Virtual memory is a technique which allows the execution of processes that may not be completely in memory. Through this technique the user program can be larger than physical memory.

Benifits:

A program would no longer be constrained by the amount of physical memory available.

It increases the degree of multiprogramming because each user program uses less amount of physical memory

Less I/O would be needed to load or swap each user program into memory.

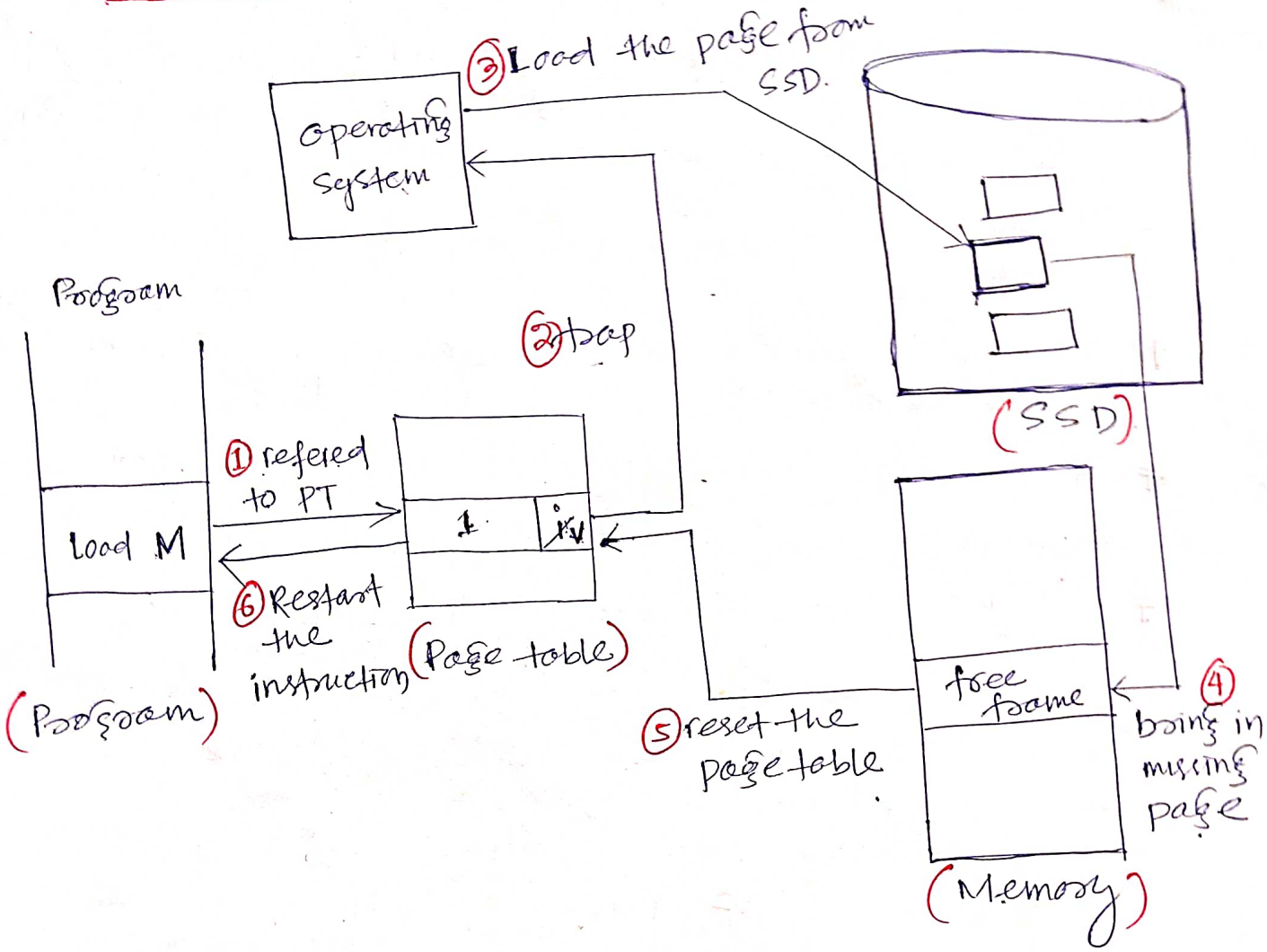
Virtual memory is commonly implimented by Demand Paging.

Demand Paging = Paging + Swapping.

With demand paging the programs are kept on the disk, when we want to execute a program it is swapped into memory but not the entire program only one page at a time.

Swapper never swaps a page into memory unless it is needed. When a program tries to use a page and that page is not in memory then a page fault occures. Page fault is served by the operating system as follows :-

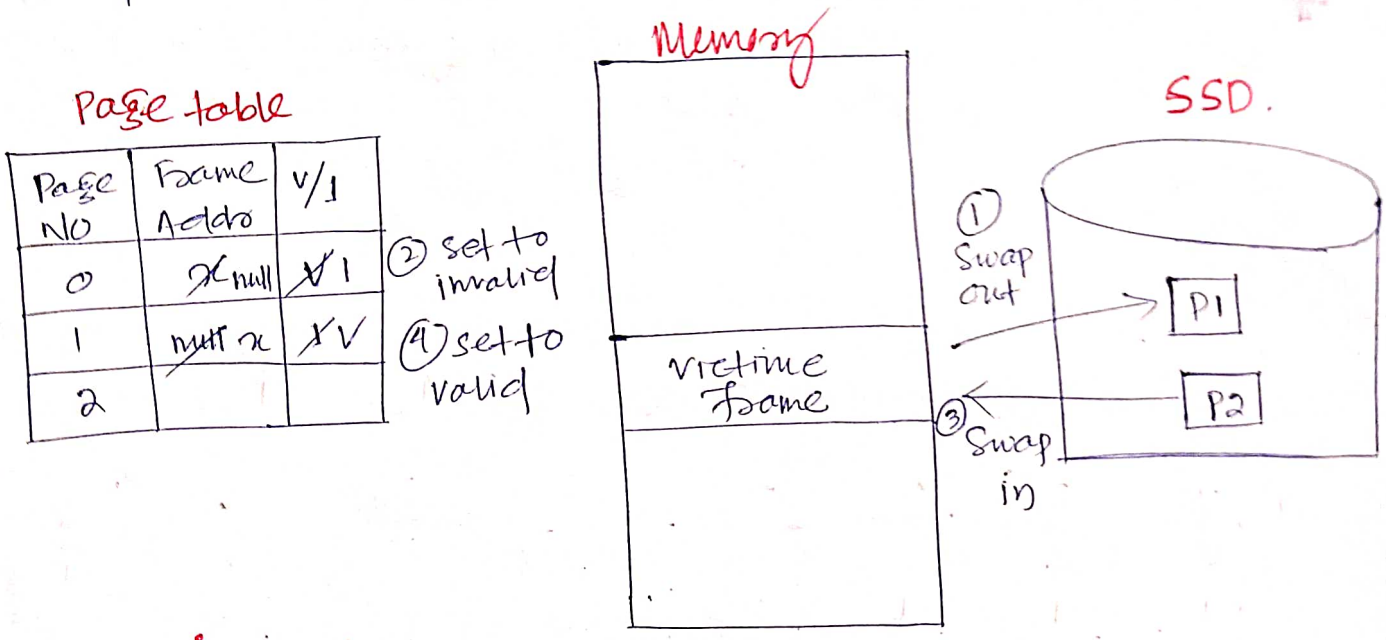
Steps in handling page fault :



In case of page faults if the os finds that there will no free frames in the list then it will use page replacement, i.e find a page that is not currently being used and free it by writting its contents back to the secondary storage and updating the page table to reflect that page is no longer available.

To reduce the page fault servie time a dirty bit can be used and it is associated with every frame This bit is set when any byte of the page is modified.

- If the page is selected for replacement its dirty bit is examined. If the dirty bit is set then only swap out is made, otherwise swap out state is avoided. This reduces the time for serving a page fault.



Steps involved in page replacement :

① Find the location of the desired page on the disk.

② Find a free frame :

a. If there is a free frame use it.

b. If there is no free frame, use a page replacement algorithm to select a victim frame

c. write the victim page to the disk; change the page and frame tables accordingly

③ Read the desired page into the newly free frame, change the page and frame tables.

④ Restart the user process.

Page Replacement Algorithms:

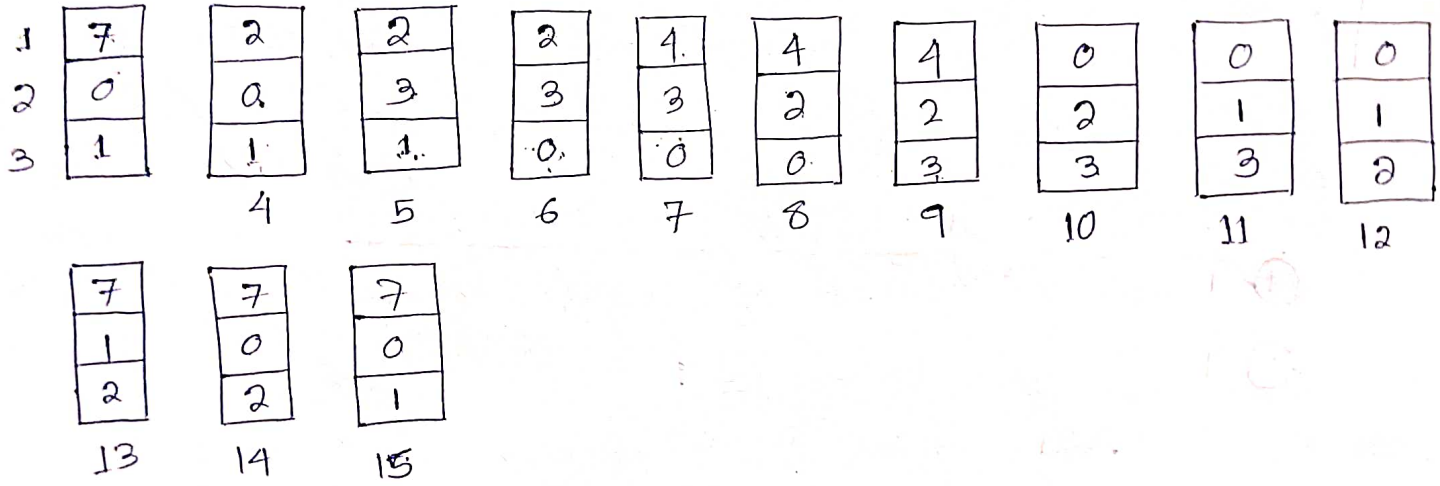
① FIFO page replacement:

The simplest page replacement algorithm is a FIFO Algo. It associates each page with the time when it was brought in memory. When a page must be replaced the oldest page is chosen as the victim.

No of allocated frame = 3

Page reference string =

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1



Advantage: It is easy to understand and impliment

Disadvantage: ① Its performance is not good because it replaces a frame which may be in constant use.

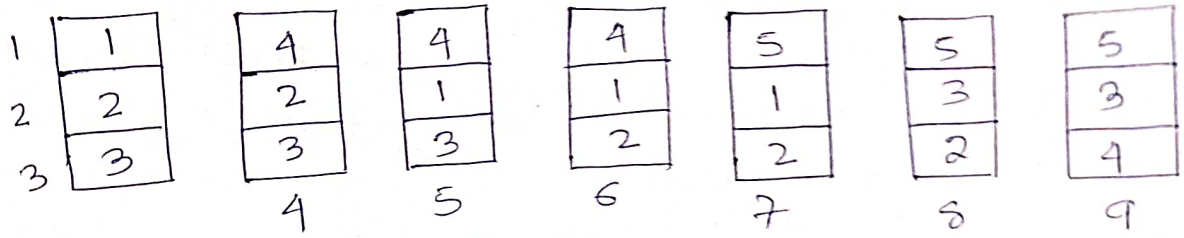
② Some times it deviates from the standard rule.

Belady's Anomaly:

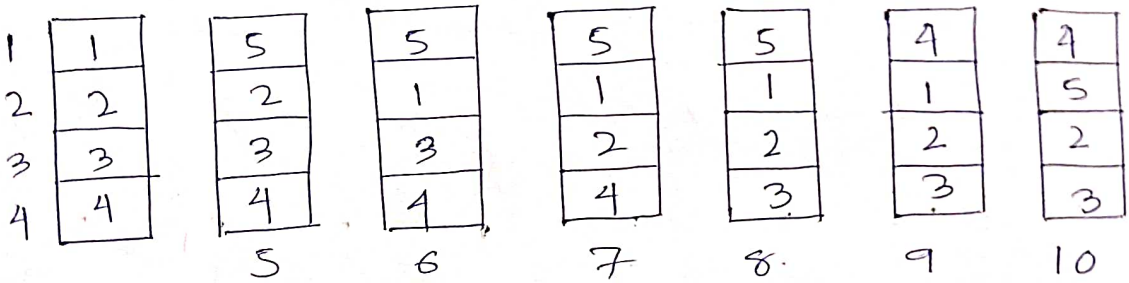
It was proved that page fault rate increases if the number of allocated frame increases. We would expect that giving more memory to a process would improve its performance. Investigation noticed that this assumption was not always true.

Page ref string : 1 2 3 4 1 2 5 1 2 3 4 5

No of frames = 3 (9)



No of frames = 4 (10)



(2) Optimal page replacement :

#An optimal page replacement algorithm has the lowest page fault rate of all algorithms and will never suffer from Belady's anomaly. Such an algorithm does exist and has been called optimal or minimal.

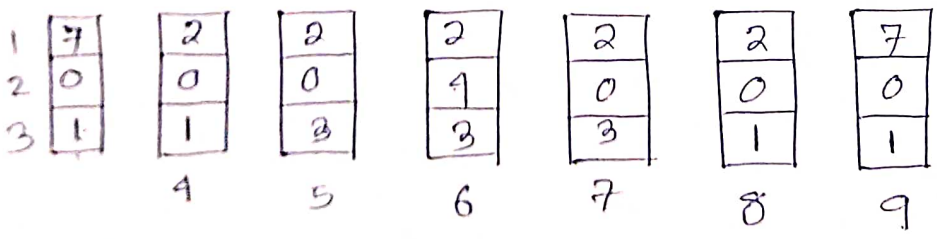
It is simply :

"Replace the page that will not be used for the longest period of time."

No of allocated frame = 3

Page reference string :

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7 0 1



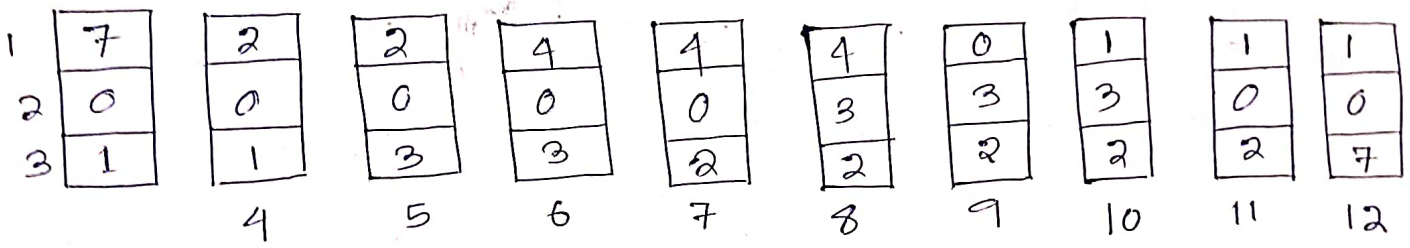
Advantage : It has the lowest fault rate.

Disadvantage : It is difficult to compliment because it requires the future knowledge of reference.

⑤ Least Recently Used (LRU) Algorithm :

Here the recent past is used as the approximation of the near future and it would replace the page which has not been used for the longest period of time. It associates each page with the time of its last use. When a page must be replaced it chooses that page which has not been used for longest period of time in past.

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7 0 1



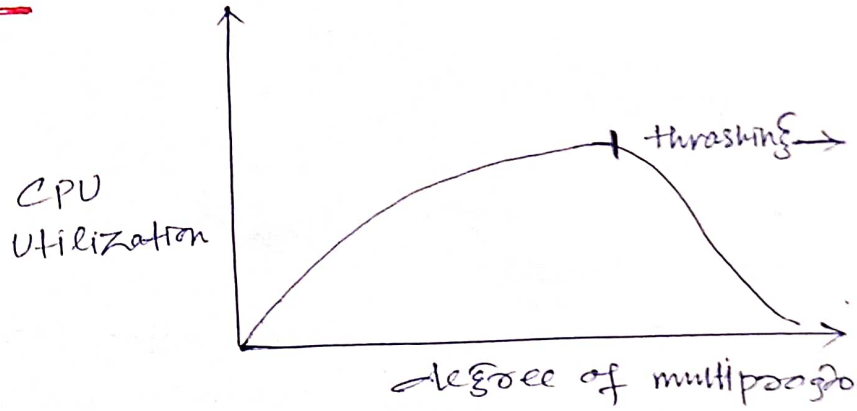
No of allocated frame = 3

No of page fault = 12

Advantage : If compare with optimal algorithm it can be implimented and provides better performance than FIFO.

Disadvantage : It is based on predicted value.

Thrashing :



Although it is possible to reduce the no of allocated frames to the minimum there will be some no of pages that will be in active use.

If the process does not have sufficient frames for active pages it will very quickly starts page fault.

It will replace these pages which will be needed again and again. Consequently page fault will increase.

This very high paging activity is known as thrashing.

A particular process is said to be thrashing if it is spending more time on paging rather than executing.

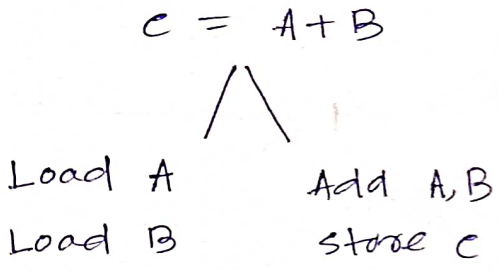
How to overcome this ?

As the DMP increases CPU Utilization increases until a maximum value. After that if we increase DMP than thrashing will set and CPU Utilization is drop sharply.

At this point OS decreases the DMP to stop thrashing.

The effect of thrashing can be minimized if a process is allocated with as many frames as needed by active no of pages.

Frame Allocation :



Minimum number of frame must be allocated to a processor. When a page fault occures before an executing instruction is complete that instruction must be restarted after serving the page fault.

Therefore A process must have enough frames to hold all different pages that any single instruction can refer to. Thus minimum no of frame is defined by the instruction set architecture of machine.

Frame Allocation Algorithm :

(1) Equal Allocation :

We can split m number of frames among n no of processes giving each process an equal share of m/n frames.

In this case some frames may be wasted if the process is too small.

(2) Proportional Allocation :

The available memory is allocated to each process according to its size.

$$a_i = \frac{s_i}{S} \times m$$

a_i = allocated to process P_i

s_i = size of process P_i

$S = \sum s_i$

m = no of frames.

How many active pages?

① **Locality model:** As a program executes it moves from locality to locality. A locality can be defined as set of pages which are actively used together. A program consists of several different localities which can overlap.
If sufficient frames are allocated for 1 locality then it will never fault again until changes locality.

② **Working set model:**

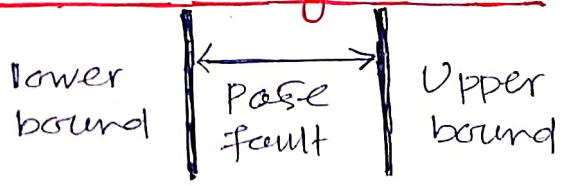
It defines a working set window which keeps the most recent page references. (Δ).

If a page ~~comes~~ ~~under~~ is in active use it comes under the working set. If a page is not under use it will be dropped from the working set after (Δt) time period of its last reference.

OS monitors the working set of each process and allocate it enough frames according to its working set size.

If there are enough extra frame another process can be initiated. If sufficient frames are not available for all process than it can select a process to suspend.

③ **Page fault frequency model:**



OS can establish the lower and upper bounds on the desired page fault rate. If the actual page-fault exceeds the upper limit then that process will be allocated with another frame.

If the page fault rate falls below the lower limit then a frame is removed from the process.

Example of working set:

2 6 1 5 7 7 7 7 5 1 6 2 3 4 1 2 3 4 4 4 3 4 3 4 4 4 1 3 2 2 3 4 4

Δ = 10

$WS_{t_1} = \{1, 2, 5, 6, 7\}$ t_1

$WS_{t_2} = \{3, 4\}$ t_2